Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Основы системного программирования»

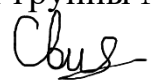
ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

«Работа с файлами и каталогами»

Вариант 3

Выполнил:

Свиржевский Алексей Денисович, студент группы N3252



(подпись)

Проверил:

Грозов Владимир Андреевич, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание 3
Задание 4
1. Описание логики программы 5
2. Makefile 6
3. Результаты тестирования 8
Вывод 10
Приложение 11

ЗАДАНИЕ

Цель работы – изучение базовых принципов системного программирования в ОС Linux, освоение системных вызовов и библиотечных функций для работы с файловой системой, а также инструментов сборки (Make), отладки и статического анализа.

Задание по варианту (нечетное количество букв в фамилии и 3 вариант):

Разработать на языке C консольную утилиту, выполняющую рекурсивный поиск файлов, начиная с указанного каталога.

Критерий поиска: файлы должны содержать заданную последовательность байтов.

Формат аргумента поиска: 0xhh[hh*] (где hh – шестнадцатеричные цифры).

Ограничение варианта: для обхода дерева каталогов необходимо использовать функцию ftw().

1. ОПИСАНИЕ ЛОГИКИ ПРОГРАММЫ

Для реализации поставленных задач в программе было выполнено:

1. Обработка аргументов: искомая строка вида 0xc0ffee валидируется и преобразуется в динамический массив байтов uint8_t. Парсинг флагов -h и -v реализован с использованием функции getopt_long().
2. Обход каталогов: выполнение с помощью функции ftw(). Для защиты от падения программы ошибки доступа к каталогам (FTW_DNR) или ошибки stat (FTW_NS) перехватываются внутри функции, и выводится предупреждение в stderr, после чего обход продолжается.
3. Алгоритм поиска файлов: чтение производится побайтово (fgetc). Если совпадение обрывается на середине искомой последовательности, программа использует fseek для отката файлового указателя на позицию начала совпадения + 1.

2. MAKEFILE

Сборка проекта и тестирование осуществляются с помощью утилиты make.

Содержимое Makefile:

```
SHELL := /bin/bash
CC = gcc
CFLAGS = -Wall -Wextra -Werror -std=c11
TARGET = lab1asdN3252
SRC = lab1.c
OBJ = $(SRC:.c=.o)

.PHONY: all clean test analyze archive

all: $(TARGET)

$(TARGET): $(OBJ)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

analyze: clean
    $(CC) $(CFLAGS) -fanalyzer $(SRC) -o $(TARGET) 2> analyzer.txt
|| true
    @echo "Статический анализ завершен. Результат в analyzer.txt"

test: $(TARGET)
    @echo -e "\n1. Подготовка тестовой среды"
    rm -rf test
    mkdir -p test/sub_dir
    mkdir -p test/no_access_dir
```

```
	printf "\xAA\xBB\xCC\xDD" > test/match1.bin
	printf "\x00\x11\xAA\xBB\xCC\xDD" > test/sub_dir/match2.bin
	printf "\xAA\xBB\xAA\xBB\xCC\xDD" > test/match3.bin
	printf "\x11\x22\x33" > test/no_match.bin
	echo "secret" > test/no_access.bin
	chmod 000 test/no_access.bin
	chmod 000 test/no_access_dir

	@echo -e "\n2. Запуск программы (Ищем 0xAABBCCDD)"
	LAB1DEBUG=1 ./$(TARGET) test 0xAABBCCDD || true

	@echo -e "\n3. Запуск под Valgrind"
	  valgrind --leak-check=full --show-leak-kinds=all --log-file=valgrind.txt ./$(TARGET) test 0xAABBCCDD || true
	@echo -e "\nОТЧЕТ VALGRIND"
	@grep "HEAP SUMMARY:" valgrind.txt -A 5 || true

	@echo -e "\n4. Восстановление прав для архивации"
	chmod 777 test/no_access.bin test/no_access_dir || true

clean:
	rm -f $(OBJ) $(TARGET)

archive: clean
	cd .. && tar -czvf $(TARGET).tar.gz $(TARGET)/
	  @echo -e "\nАрхив $(TARGET).tar.gz успешно создан в родительской директории."
```

3. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

make test (рис. 1):

```
┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
└─$ make test

1. Подготовка тестовой среды
rm -rf test_dir
mkdir -p test_dir/sub_dir
mkdir -p test_dir/no_access_dir
printf "\xAA\xBB\xCC\xDD" > test_dir/match1.bin
printf "\x00\x11\xAA\xBB\xCC\xDD" > test_dir/sub_dir/match2.bin
printf "\xAA\xBB\xAA\xBB\xCC\xDD" > test_dir/match3.bin
printf "\x11\x22\x33" > test_dir/no_match.bin
echo "secret" > test_dir/no_access.bin
chmod 000 test_dir/no_access.bin
chmod 000 test_dir/no_access_dir

2. Запуск программы (Ищем 0xAABBCCDD)
LAB1DEBUG=1 ./lab1asdN3252 test_dir 0xAABBCCDD || true
[LAB1DEBUG] Запуск поиска в каталоге: test_dir
[LAB1DEBUG] Искомая последовательность байтов: AA BB CC DD
[LAB1DEBUG] Файл 'test_dir/match1.bin' соответствует критерию. Совпадение найдено по смещению: 0x0
test_dir/match1.bin
[LAB1DEBUG] Файл 'test_dir/match3.bin' соответствует критерию. Совпадение найдено по смещению: 0x2
test_dir/match3.bin
[LAB1DEBUG] Файл 'test_dir/sub_dir/match2.bin' соответствует критерию. Совпадение найдено по смещению: 0x2
test_dir/sub_dir/match2.bin

3. Запуск под Valgrind
valgrind --leak-check=full --show-leak-kinds=all --log-file=valgrind.txt ./lab1asdN3252 test_dir 0xAABBCCDD || true
test_dir/match1.bin
test_dir/match3.bin
test_dir/sub_dir/match2.bin

ОТЧЕТ VALGRIND
==786== HEAP SUMMARY:
==786==     in use at exit: 0 bytes in 0 blocks
==786==   total heap usage: 22 allocs, 22 frees, 126,692 bytes allocated
==786==
==786== All heap blocks were freed -- no leaks are possible
==786==

  4. Восстановление прав для очистки
  chmod 777 test_dir/no_access.bin test_dir/no_access_dir || true

  valgrind --leak-check=full --show-leak-kinds=all --log-file=valgrind.txt ./lab1asdN3252 test_dir 0xAABBCCDD || true
  test_dir/match1.bin
  test_dir/match3.bin
  test_dir/sub_dir/match2.bin

  ОТЧЕТ VALGRIND
  ==786== HEAP SUMMARY:
  ==786==     in use at exit: 0 bytes in 0 blocks
  ==786==   total heap usage: 22 allocs, 22 frees, 126,692 bytes allocated
  ==786==
  ==786== All heap blocks were freed -- no leaks are possible
  ==786==

  4. Восстановление прав для очистки
  chmod 777 test_dir/no_access.bin test_dir/no_access_dir || true

  ┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
  └─$
```

Рисунок 1.

```
┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
└─$ ./lab1asdN3252 --help
Использование: ./lab1asdN3252 [опции] <каталог> <цель_поиска>
Опции:
  -h, --help     Вывод справки по опциям.
  -v, --version  Вывод версии программы и информации об авторе.

Формат цели_поиска: 0xhh[hh*] (например, 0xc0ffee)
```

Рисунок 2 – Тест №2

```
┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
└─$ ./lab1asdN3252 -v
Версия 1.0
Выполнил: Свиржевский А.Д.
Группа: N3252
Вариант: 3 (ftw)
```

Рисунок 3 – Тест №3

```
┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
└─$ ./lab1asdN3252 /home 0xZZ
Ошибка: недопустимый шестнадцатеричный байт 'ZZ'.
```

Рисунок 4 – Тест №4

```
┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
└─$ ./lab1asdN3252 /home AABBCC
Ошибка: аргумент цель_поиска должен начинаться с префикса 0x.
```

Рисунок 5 – Тест №5

```
┌──(nolis㉿dekalexey)-[/mnt/c/My Projects/OSP/lab1asdN3252]
└─$ ./lab1asdN3252 /home 0xc0ffee 0xc0ffee
Ошибка: неверное количество аргументов.
Использование: ./lab1asdN3252 [опции] <каталог> <цель_поиска>
Опции:
  -h, --help     Вывод справки по опциям.
  -v, --version  Вывод версии программы и информации об авторе.

Формат цели_поиска: 0xhh[hh*] (например, 0xc0ffee)
```

Рисунок 6 – Тест №6

ВЫВОД

В ходе выполнения лабораторной работы была успешно разработана утилита для поиска последовательности байтов в файлах. Были изучены механизмы работы с функцией ftw() для обхода дерева каталогов. Навыки автоматизации тестирования были закреплены при помощи утилиты GNU Make. Все задачи выполнены.

ПРИЛОЖЕНИЕ

Исходный код программы lab1asdN3252.c:

```
#define _XOPEN_SOURCE 500

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <stdint.h>
#include <errno.h>
#include <ftw.h>
#include <getopt.h>
#include <sys/stat.h>

uint8_t *search_bytes = NULL;
size_t search_len = 0;
bool debug_mode = false;

void print_help(const char *prog_name) {
    printf("Использование: %s [опции] <каталог> <цель_поиска>\n",
prog_name);
    printf("Опции:\n");
    printf("  -h, --help     Вывод справки по опциям.\n");
    printf("  -v, --version  Вывод версии программы и информации
об авторе.\n");
        printf("\nФормат   цели_поиска:   0xhh[hh*]   (например,
0xc0ffee)\n");
}

void print_version(void) {
      printf("Версия  1.0\nВыполнил:  Свиржевский  А.Д.\nГруппа:
N3252\nВариант: 3 (ftw)\n");
```

```
    }

    bool parse_hex_string(const char *hex_str) {
        if (strncmp(hex_str, "0x", 2) != 0 && strncmp(hex_str, "0X",
2) != 0) {
            fprintf(stderr, "Ошибка: аргумент цель_поиска должен
начинаться с префикса 0x.\n");
            return false;
        }
        hex_str += 2;
        size_t len = strlen(hex_str);
        if (len == 0 || len % 2 != 0) {
            fprintf(stderr, "Ошибка: после 0x должно быть четное
количество шестнадцатеричных цифр.\n");
            return false;
        }
        search_len = len / 2;
        search_bytes = (uint8_t*)malloc(search_len*sizeof(uint8_t));
        if (!search_bytes) {
            perror("Ошибка выделения памяти (malloc)");
            return false;
        }
        for (size_t i = 0; i < search_len; i++) {
            char byte_str[3] = {hex_str[2*i], hex_str[2*i + 1], '\0'
};
            char *endptr;
            long val = strtol(byte_str, &endptr, 16);

            if (*endptr != '\0' || val < 0 || val > 255) {
                fprintf(stderr, "Ошибка: недопустимый шестнадцатеричный
байт '%s'.\n", byte_str);
                free(search_bytes);
                search_bytes = NULL;
```

```
                return false;
            }
            search_bytes[i] = (uint8_t)val;
        }
        return true;
    }

    bool search_in_file(const char *fpath) {
        FILE *f = fopen(fpath, "rb"); // Открываем в бинарном режиме
        if (!f) {
             // Ошибка доступа к файлу — выводим сообщение в поток ошибок и продолжаем работу
            fprintf(stderr, "Ошибка доступа к файлу '%s': %s\n", fpath, strerror(errno));
            return false;
        }

        int c;
        size_t match_idx = 0;
        long match_offset = -1;
        long current_offset = 0;

        // Читаем файл побайтово
        while ((c = fgetc(f)) != EOF) {
            if ((uint8_t)c == search_bytes[match_idx]) {
                if (match_idx == 0) {
                   match_offset = current_offset; // Запоминаем начало совпадения
                }
                match_idx++;

                if (match_idx == search_len) {
                    fclose(f);
```

```
                    if (debug_mode) {
                            fprintf(stderr, "[LAB1DEBUG] Файл '%s'
соответствует критерию. "
                                    "Совпадение найдено по смещению:
0x%lX\n", fpath, match_offset);
                    }
                    return true;
                }
            } else {
                if (match_idx > 0) {
                      // Если совпадение сорвалось посередине, нужно
вернуться назад.
                    // Пример: ищем 0xAA 0xAA 0xBB в файле "0xAA 0xAA
0xAA 0xBB".
                    if (fseek(f, match_offset + 1, SEEK_SET) != 0) {
                           fprintf(stderr, "Ошибка позиционирования в
файле '%s': %s\n", fpath, strerror(errno));
                        fclose(f);
                        return false;
                    }
                    current_offset = match_offset;
                    match_idx = 0;
                }
            }
            current_offset++;
        }
        if (ferror(f)) {
             fprintf(stderr, "Ошибка чтения файла '%s': %s\n", fpath,
strerror(errno));
            clearerr(f);
        }

        fclose(f);
```

```
        return false;
    }

    int ftw_callback(const char *fpath, const struct stat *sb, int typeflag) {
          (void)sb; // Подавляем предупреждение о неиспользуемом параметре
        if (typeflag == FTW_DNR) {
            fprintf(stderr, "Ошибка доступа к директории '%s': %s\n", fpath, strerror(errno));
            return 0; // Продолжать обход
        }
        if (typeflag == FTW_NS) {
            fprintf(stderr, "Ошибка получения информации о файле '%s': %s\n", fpath, strerror(errno));
            return 0; // Продолжать обход
        }
        if (typeflag == FTW_F) {
            if (search_in_file(fpath)) {
                printf("%s\n", fpath);
            }
        }
        return 0; // Продолжать обход
    }

    int main(int argc, char *argv[]) {
        if (getenv("LAB1DEBUG") != NULL) {
            debug_mode = true;
        }

        static struct option long_options[] = {
            {"help", no_argument, 0, 'h'},
            {"version", no_argument, 0, 'v'},
```

```
            {0, 0, 0, 0}
        };
        int opt;
         while ((opt = getopt_long(argc, argv, "vh", long_options,
NULL)) != -1) {
            switch (opt) {
                case 'v':
                    print_version(); // Вызываем вывод версии
                    return 0;
                case 'h':
                    print_help(argv[0]); // Вызываем вывод справки
                    return 0;
                default:
                      fprintf(stderr, "Неверная опция. Используйте --
help для справки.\n");
                    return 1;
            }
        }
        if (optind + 2 != argc) {
                    fprintf(stderr,   "Ошибка:   неверное   количество
аргументов.\n");
            print_help(argv[0]);
            return 1;
        }

        const char *start_dir = argv[optind];
        const char *hex_str = argv[optind + 1];
        if (!parse_hex_string(hex_str)) {
            return 1;
        }

        if (debug_mode) {
```

```
            fprintf(stderr, "[LAB1DEBUG] Запуск поиска в каталоге: %s\n", start_dir);
            fprintf(stderr, "[LAB1DEBUG] Искомая последовательность байтов: ");
            for (size_t i = 0; i < search_len; i++) {
                fprintf(stderr, "%02X ", search_bytes[i]);
            }
            fprintf(stderr, "\n");
        }

        int nopenfd = 20;
        if (ftw(start_dir, ftw_callback, nopenfd) == -1) {
            fprintf(stderr, "Фатальная ошибка при обходе дерева каталогов (ftw): %s\n", strerror(errno));
            free(search_bytes);
            return 1;
        }

        free(search_bytes);
        return 0;
    }
```